

Examen Parcial 2

About Dataset

Machine Predictive Maintenance Classification Dataset Since real predictive maintenance datasets are generally difficult to obtain and in particular difficult to publish, we present and provide a synthetic dataset that reflects real predictive maintenance encountered in the industry to the best of our knowledge.

The dataset consists of 10 000 data points stored as rows with 14 features in columns

- UID: unique identifier ranging from 1 to 10000
- productID: consisting of a letter L, M, or H for low (50% of all products), medium (30%), and high (20%) as product quality variants and a variant-specific serial number
- air temperature [K]: generated using a random walk process later normalized to a standard deviation of 2 K around 300 K
- process temperature [K]: generated using a random walk process normalized to a standard deviation of 1 K, added to the air temperature plus 10 K.
- rotational speed [rpm]: calculated from powepower of 2860 W, overlaid with a normally distributed noise
- torque [Nm]: torque values are normally distributed around 40 Nm with an $\sigma = 10$ Nm and no negative values.
- tool wear [min]: The quality variants H/M/L add 5/3/2 minutes of tool wear to the used tool in the process. and a
- 'machine failure' label that indicates, whether the machine has failed in this particular data point for any of the following failure modes are true.

Important :

There are two Targets - Do not make the mistake of using one of them as feature, as it will lead to leakage.

- Target : Failure or Not
- Failure Type : Type of Failure

Nota importante: Para este examen vamos a utilizar la variable objetivo, **Target** es binaria si la maquina fallo o no, la otra variable hay que eliminarla ya que puede causar data leakage

Pregunta 1: Investigar y explicar en un parrafo en formato markdown que es el **data leakage** y como esta puede afectar a nuestro modelo de ML, especificamente en este caso porque hay que eliminar una de las variables objetivo

Data leakage o fuga de datos es cuando en el modelo de Machine Learning se usa información que en la vida real no se tendría al momento de hacer la predicción. Esto puede hacer que el modelo parezca muy bueno entrenando, pero en realidad está aprendiendo con datos que no debería usar, y por eso después no funciona bien con datos nuevos. En este caso, tenemos dos variables objetivo: **Target** , que nos dice si la máquina falló o no, y **Failure Type** , que nos dice qué tipo de falla fue. Si usamos las dos, el modelo estaría "viendo" el tipo de falla antes de saber si hubo una o no, lo que hace que aprenda con trampa. Por eso, se debe eliminar **Failure Type** y quedarnos solo con **Target** , así el modelo no se contamina con información futura.

Carga de datos

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	No Failure
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	No Failure
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	No Failure
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	No Failure
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	No Failure
...	...	...	...	...	...	...	...	...	...	...
9995	9996	M24855	M	298.8	308.4	1604	29.5	14	0	No Failure
9996	9997	H39410	H	298.9	308.4	1632	31.8	17	0	No Failure
9997	9998	M24857	M	299.0	308.6	1645	33.4	22	0	No Failure
9998	9999	H39412	H	299.0	308.7	1408	48.5	25	0	No Failure
9999	10000	M24859	M	299.0	308.7	1500	40.2	30	0	No Failure

10000 rows × 10 columns

Preprocesamiento de datos

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   UDI                   10000 non-null  int64
 1   Product ID           10000 non-null  object
 2   Type                 10000 non-null  object
 3   Air temperature [K]  10000 non-null  float64
 4   Process temperature [K] 10000 non-null  float64
 5   Rotational speed [rpm] 10000 non-null  int64
 6   Torque [Nm]          10000 non-null  float64
 7   Tool wear [min]      10000 non-null  int64
 8   Target               10000 non-null  int64
 9   Failure Type         10000 non-null  object
dtypes: float64(3), int64(4), object(3)
memory usage: 781.4+ KB

```

Validacion de datos Null o duplicados

```

UDI                0
Product ID         0
Type               0
Air temperature [K] 0
Process temperature [K] 0
Rotational speed [rpm] 0
Torque [Nm]        0
Tool wear [min]    0
Target             0
Failure Type       0
dtype: int64

```

```
np.int64(0)
```

Entendiendo los tipos de datos y valores unicos

```

UDI                int64
Product ID         object
Type               object
Air temperature [K] float64
Process temperature [K] float64
Rotational speed [rpm] int64
Torque [Nm]        float64
Tool wear [min]    int64
Target             int64
Failure Type       object
dtype: object

```

```
UDI                10000
Product ID        10000
Type               3
Air temperature [K] 93
Process temperature [K] 82
Rotational speed [rpm] 941
Torque [Nm]        577
Tool wear [min]    246
Target            2
Failure Type       6
dtype: int64
```

```
Type
L    6000
M    2997
H    1003
Name: count, dtype: int64
```

Pregunta 2: Que entiendes de el analisis anterior

Al revisar los resultados anteriores, se puede ver que el dataset está limpio, ya que no hay valores nulos ni filas repetidas. También se identificaron correctamente los tipos de datos, entre numéricos y categóricos, lo cual es útil para las siguientes etapas del análisis.

Se observó que columnas como `UDI` y `Product ID` tienen valores únicos en cada fila, lo que indica que son identificadores. Como estos no aportan información útil para el modelo, lo mejor es descartarlos.

Además, en la variable `Type`, que representa el tipo de producto, se encontraron tres clases: L, M y H. Sin embargo, no están balanceadas: la mayoría de los registros son del tipo L. Este desbalance podría afectar el entrenamiento del modelo, ya que puede sesgar los resultados hacia esa clase más frecuente.

Eliminar identificadores

Elimina las variables

- `UDI`
- `Product ID`
- `Failure Type`

Tip: utiliza la función `drop` de pandas

```
['UDI', 'Product ID', 'Type', 'Air temperature [K]', 'Process temperature [K]',  
'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]', 'Target', 'Failure  
Type']
```

Verificamos si el dataset esta balanceado

Tip: Utiliza la funcion de pandas `value_counts()`

```
Target
0      9661
1       339
Name: count, dtype: int64
```

Pregunta 3: Esta balanceado o no el dataset, que estrategias hay para un dataset no balanceado

El dataset no está balanceado, ya que hay muchas más muestras con clase 0 (9661) que con clase 1 (339). Esto significa que la mayoría de los datos son de casos donde la máquina no falló, y muy pocos de cuando sí falló. Este desbalance puede hacer que el modelo se enfoque solo en predecir la clase mayoritaria, y no aprenda bien a detectar las fallas.

Para solucionar este problema, se pueden aplicar algunas estrategias como:

- **Oversampling:** duplicar o generar ejemplos nuevos de la clase minoritaria (por ejemplo, con SMOTE).
- **Undersampling:** reducir la cantidad de muestras de la clase mayoritaria.
- **Cambiar la métrica de evaluación:** usar métricas como F1-score, recall o matriz de confusión en lugar de solo accuracy.
- **Ajustar pesos en el modelo:** darle mayor importancia a la clase minoritaria durante el entrenamiento.

Estas técnicas ayudan a que el modelo no se sesgue por la clase que aparece más.

Creamos dos variables nuevas

- temperature_difference: diferencia entre la temperatura de el aire y la del proceso
- Mechanical Power (W):

$$P = \frac{T \cdot 2\pi \cdot N}{60}$$

Donde:

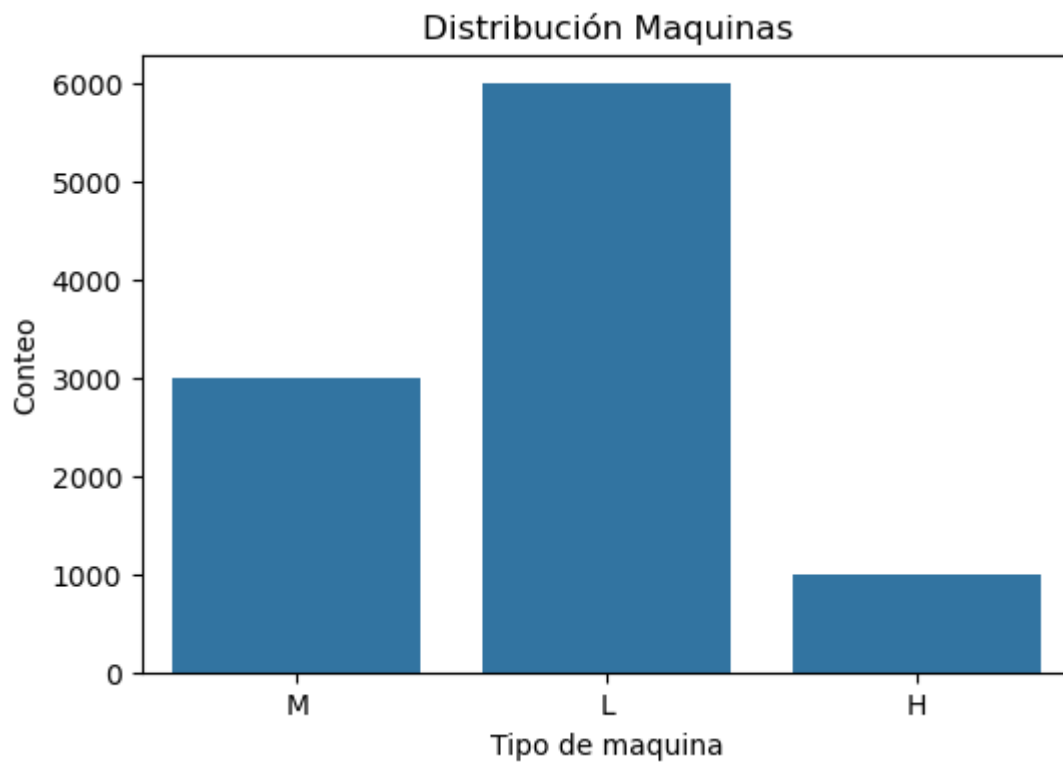
- P es la potencia en W,
- T es el torque en Nm,
- N es la velocidad en rpm,
- $\frac{2\pi}{60}$ convierte rpm a rad/s.

Aplicandolo a la data: $\text{Potencia (W)} = \frac{\text{Torque (Nm)}}{9.5493} \cdot \text{Velocidad (rpm)}$

Analisis de Datos Exploratorio

Visualización de Tipos de maquina

Grafica la distribución de tipos de maquina



Visualización de la distribución de fallos entre tipos de productos

Muestra cómo se distribuyen los fallos entre los tipos: ¿es posible que algunos tipos de maquina fallen más?

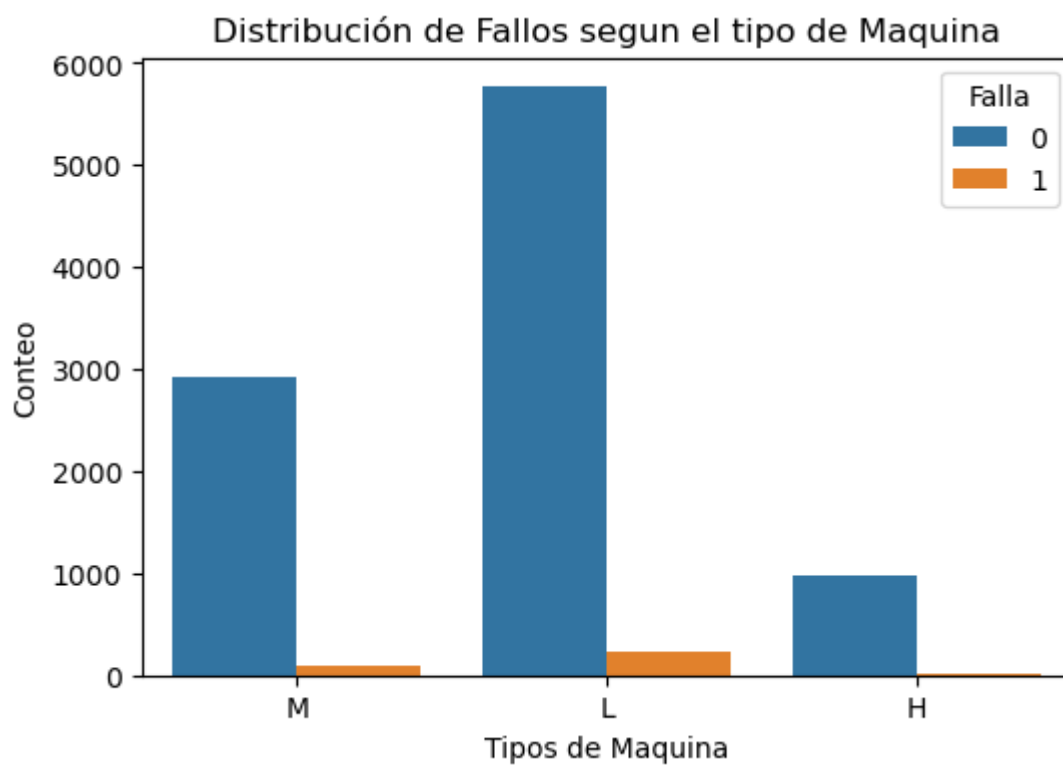
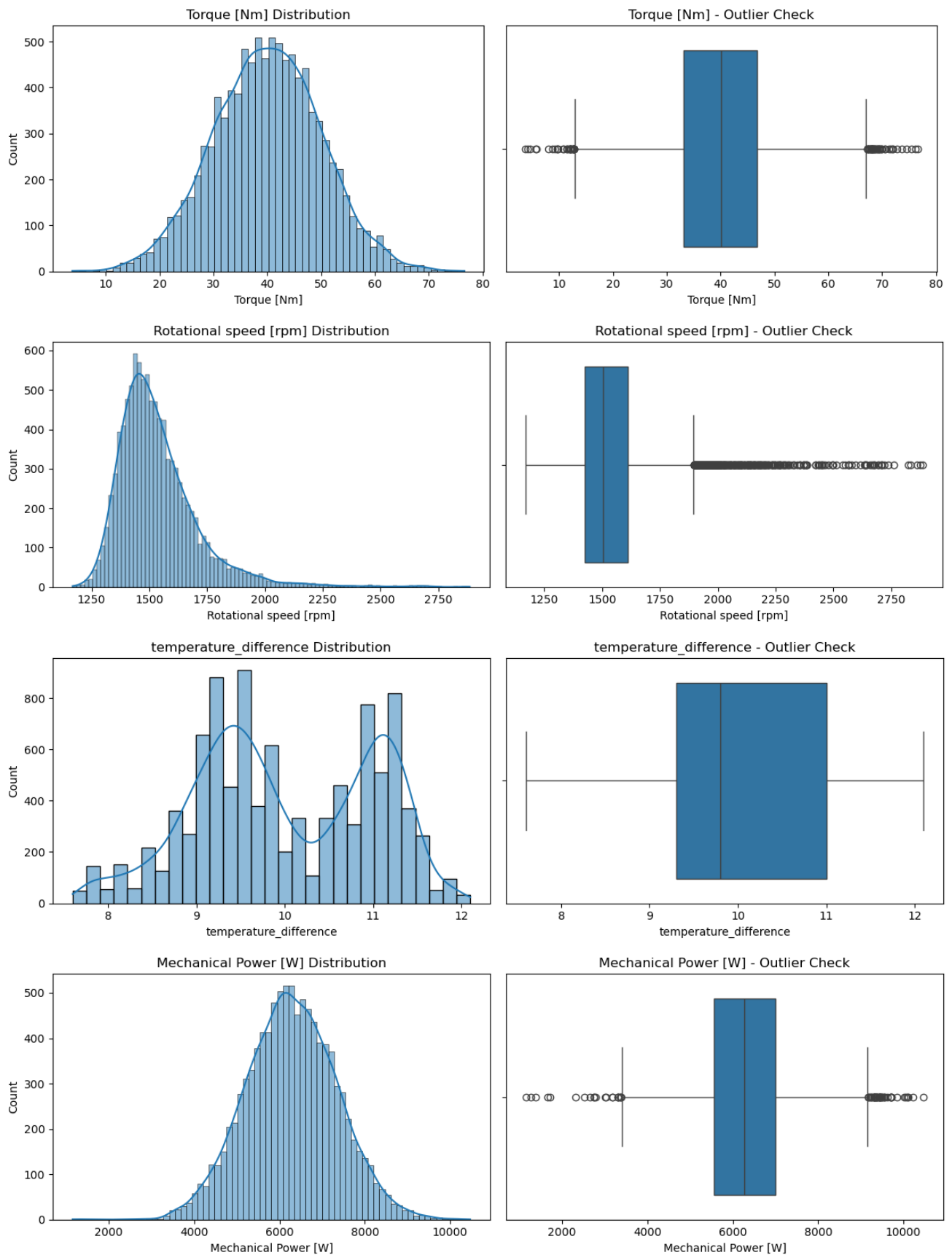
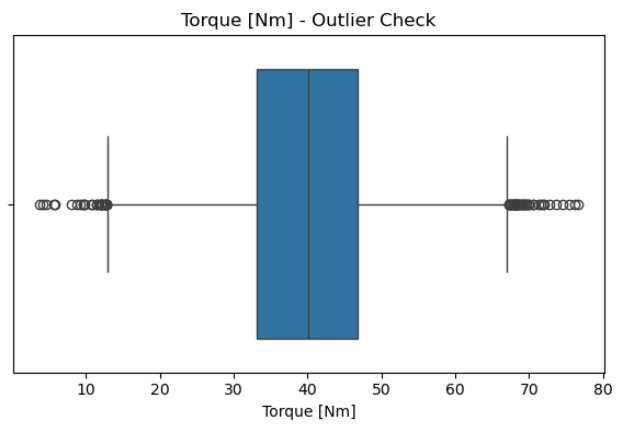
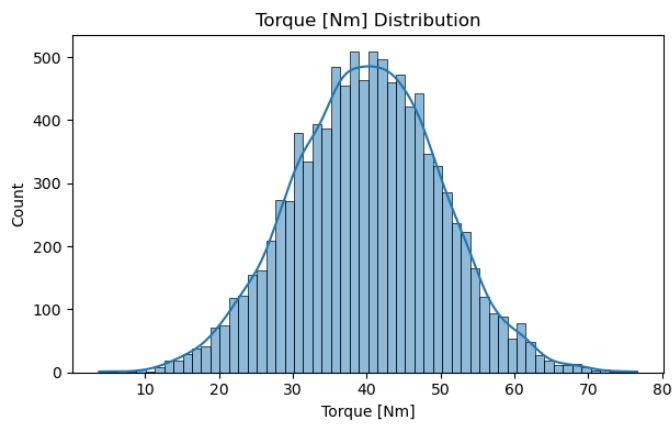
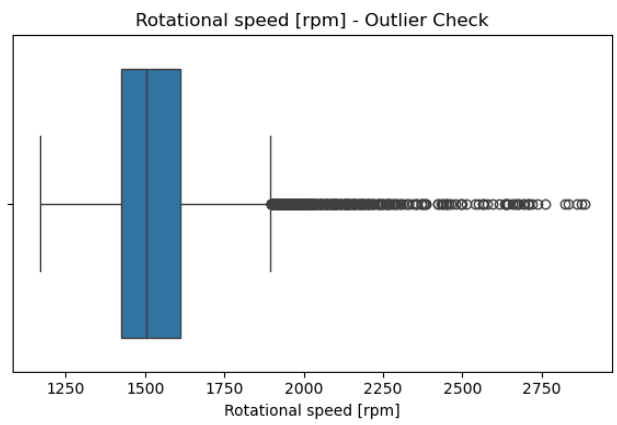
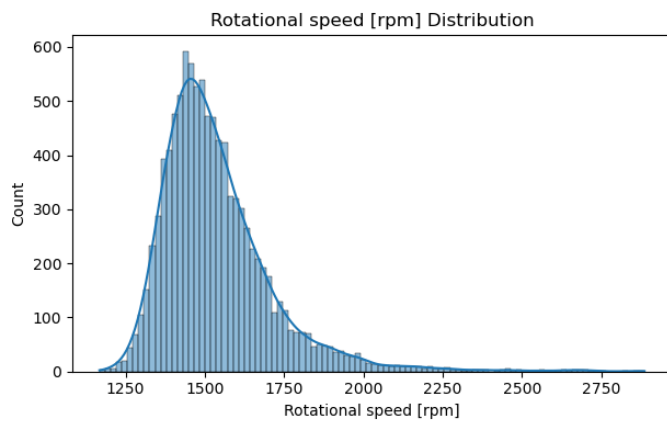


Grafico de distribuciones

Graficamos las distribuciones de las variables numericas para observar anomalias y patrones en los datos.





Pregunta 4: Describe que entiendes de la grafica anterior

Torque [Nm]

- **Histograma:** La distribución se ve bastante centrada, parecida a una campana. Los valores más comunes están entre 30 y 50 Nm, con un pico notorio cerca de 40 Nm.
- **Boxplot:** Aunque la mayoría de los datos están agrupados, aparecen varios valores que se salen del rango, tanto por arriba como por abajo. Sería bueno revisarlos antes de entrenar un modelo.

Rotational speed [rpm]

- **Histograma:** Aquí la gráfica se inclina hacia la derecha, es decir, hay más datos en los valores bajos (1300–1700 rpm) y luego se va extendiendo.
- **Boxplot:** Se identifican muchos datos atípicos por el lado alto. Aunque el centro está claro, esos outliers podrían afectar el comportamiento del modelo si no se manejan.

temperature_difference

- **Histograma:** Tiene dos picos, lo que sugiere que hay dos grupos distintos en esta variable. Puede que estén representando condiciones diferentes del proceso.
- **Boxplot:** No se observan valores fuera de rango, lo que significa que, aunque haya dos grupos, todos los datos están dentro de un margen razonable.

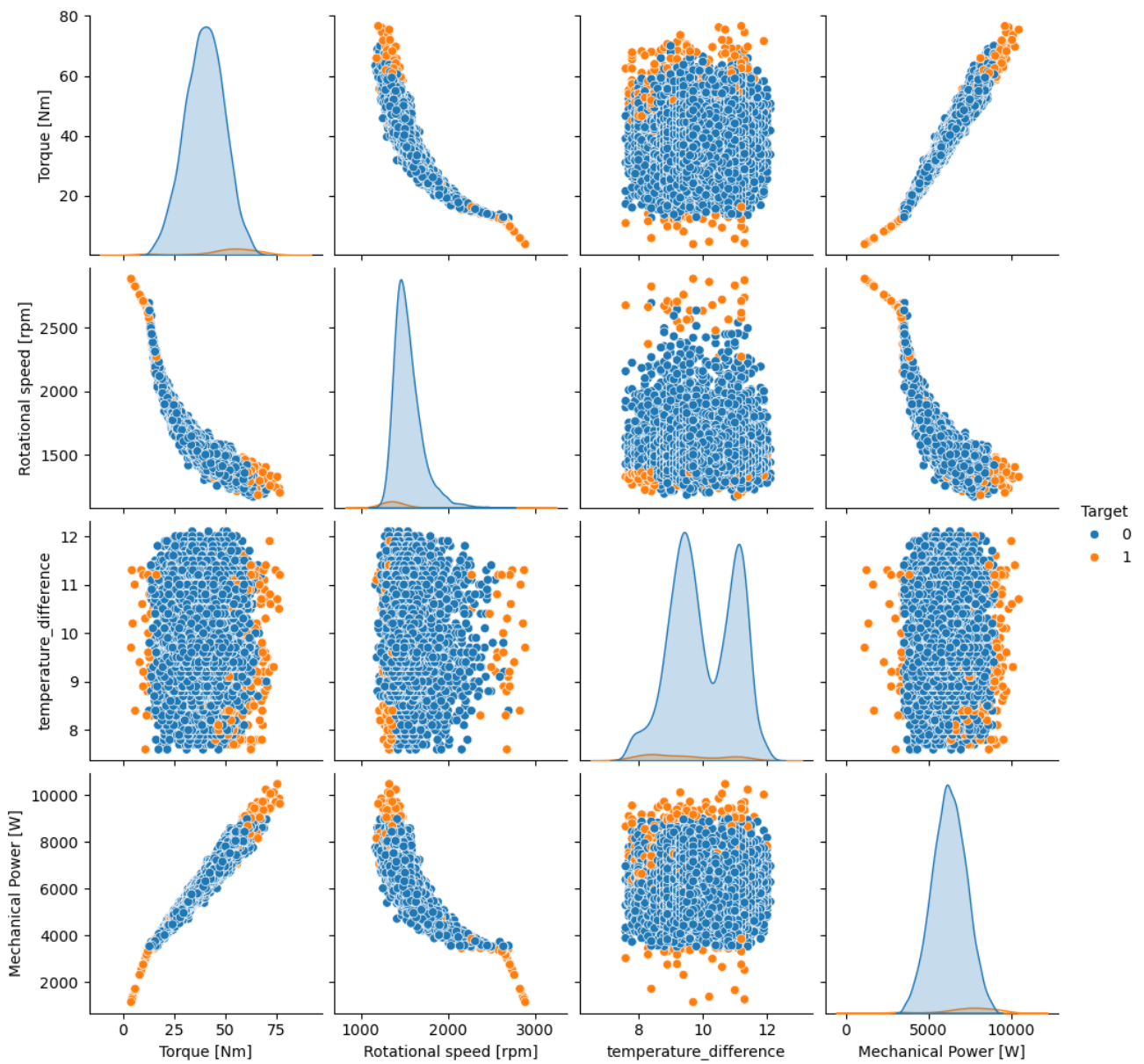
Mechanical Power [W]

- **Histograma:** También parece una campana, centrada entre 5000 y 7000 W, lo cual es típico de una distribución normal.
- **Boxplot:** Aunque se ve estable, hay algunos valores extremos. Son pocos, pero igual conviene tenerlos en cuenta si se va a hacer un modelo predictivo.

Resumen general: En general, las variables tienen formas bastante limpias y estables. Algunas como `rotational speed` o `mechanical power` presentan valores atípicos que podrían complicar los resultados si no se tratan. En cambio, `temperature_difference` resalta por su doble pico, lo que nos dice que podrían existir dos escenarios distintos en el proceso. Estos aspectos deben analizarse bien antes de aplicar cualquier modelo de aprendizaje automático, ya que pueden influir mucho en la precisión del resultado.

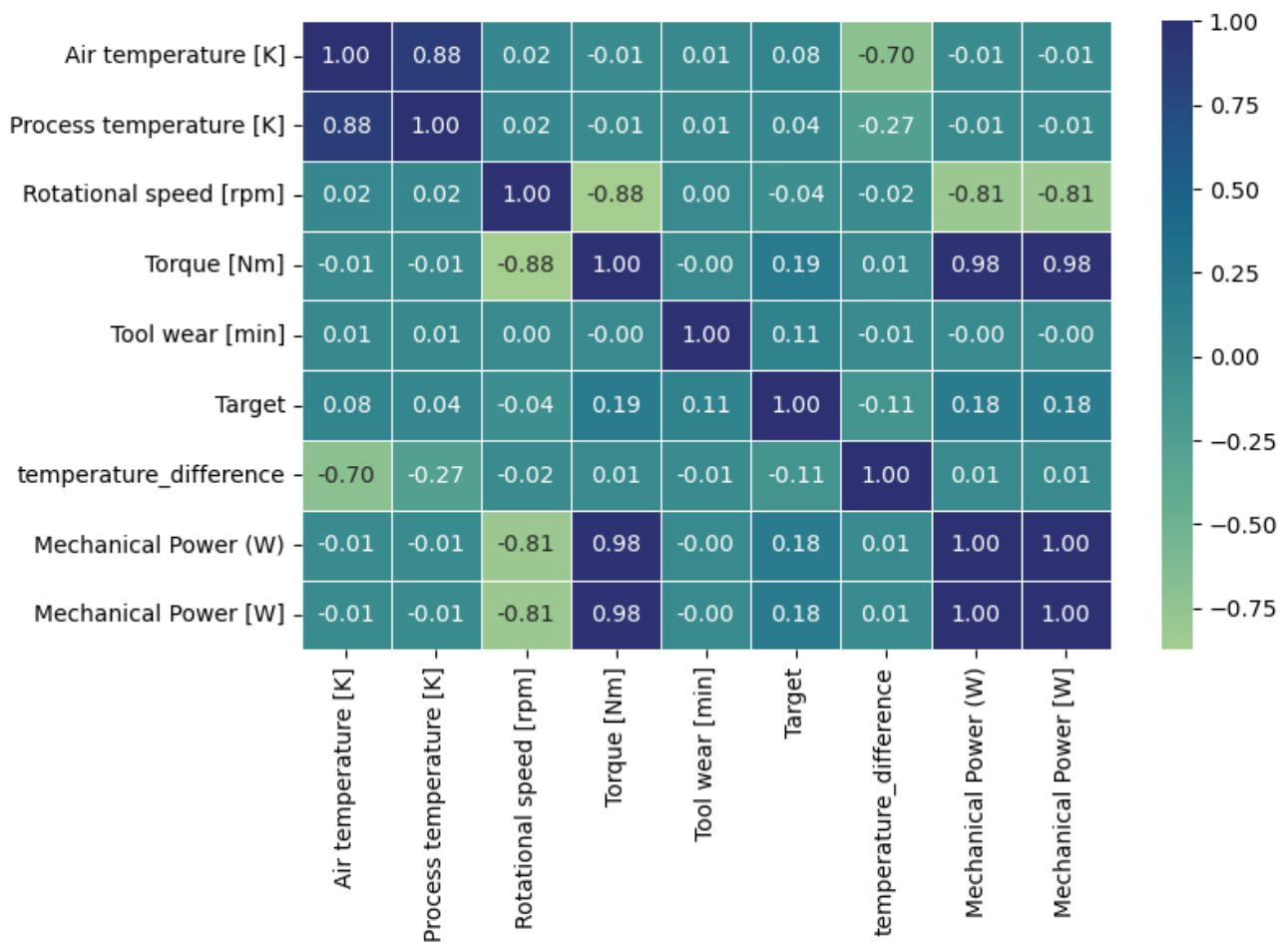
Diagrama de pares para relaciones de características¶

Muestra la interacción entre las características coloreadas por falla.



Correlacion entre variables numericas

<Axes: >



Pregunta 5: Investiga sobre la matriz de confusión, que interpretas de estos resultados

Matriz de confusión y análisis de relaciones

¿Qué es una matriz de confusión?

La matriz de confusión es una herramienta usada para evaluar el rendimiento de un modelo de clasificación. Muestra la cantidad de predicciones correctas e incorrectas organizadas por clases reales y predichas. Nos permite identificar cuántas veces el modelo acertó (verdaderos positivos y negativos) y cuántas veces se equivocó (falsos positivos y negativos), brindando una visión más detallada que una simple métrica de precisión.

Análisis de la matriz de correlación y el gráfico de pares

Diagrama de pares (Pairplot):

Este gráfico nos permite visualizar cómo interactúan entre sí varias características del dataset y cómo se relacionan con la variable objetivo (`Target`).

- La relación entre **Torque** y **Rotational speed** es inversa, lo cual se refleja claramente: a mayor torque, la velocidad rotacional disminuye.
- **Mechanical Power** tiene una relación lineal directa con el torque, como se espera debido a su fórmula.
- La variable **temperature_difference** muestra una distribución más dispersa, lo que sugiere que podría no estar tan directamente relacionada con el evento de fallo.

Matriz de correlación (Heatmap):

Nos ayuda a ver la fuerza de relación lineal entre variables numéricas:

- `Torque` y `Mechanical Power` tienen una alta correlación positiva (≈ 0.98).
- `Rotational speed` tiene una alta correlación negativa con `Torque` (≈ -0.88) y con `Mechanical Power` (≈ -0.81).
- Las variables de temperatura (`Air` y `Process`) están fuertemente correlacionadas entre sí (≈ 0.88), lo cual es lógico dado que una deriva de la otra.
- La variable `Target` presenta correlaciones débiles con las demás, lo cual es adecuado para un problema de clasificación, ya que indica que ninguna sola variable domina completamente la predicción.

Entrenando el Modelo

Separando el dataset

```

Collecting scikit-learn
  Downloading scikit_learn-1.6.1-cp310-cp310-win_amd64.whl.metadata (15 kB)
Requirement already satisfied: numpy>=1.19.5 in c:
\users\josue\anaconda3\envs\laboratorio_rd_1_jm\lib\site-packages (from scikit-
learn) (2.2.2)
Requirement already satisfied: scipy>=1.6.0 in c:
\users\josue\anaconda3\envs\laboratorio_rd_1_jm\lib\site-packages (from scikit-
learn) (1.15.1)
Collecting joblib>=1.2.0 (from scikit-learn)
  Downloading joblib-1.5.0-py3-none-any.whl.metadata (5.6 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Downloading scikit_learn-1.6.1-cp310-cp310-win_amd64.whl (11.1 MB)
----- 0.0/11.1 MB ? eta -:--:--
----- 1.3/11.1 MB 13.4 MB/s eta 0:00:01
----- 9.7/11.1 MB 31.8 MB/s eta 0:00:01
----- 11.1/11.1 MB 30.2 MB/s eta 0:00:00
Downloading joblib-1.5.0-py3-none-any.whl (307 kB)
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Installing collected packages: threadpoolctl, joblib, scikit-learn
Successfully installed joblib-1.5.0 scikit-learn-1.6.1 threadpoolctl-3.6.0

```

Utiliza la función `train_test_split` de sklearn utiliza 20% de los datos para el test set

Codificación de variables categoricas

Modelos de ML

```

===== Training model: Logistic Regression =====
Test data ROC AUC: 0.93
===== Training model: Logistic Regression CV =====
Test data ROC AUC: 0.92
===== Training model: SGD =====
Test data ROC AUC: 0.92
===== Training model: Random Forest =====
Test data ROC AUC: 0.96
===== Training model: Gradient Boosting =====
Test data ROC AUC: 0.98
===== Training model: Decision Tree =====
Test data ROC AUC: 0.88

```

Model Performance:

```
===== Model: Gradient Boosting =====
Precision: 0.964286
Recall:    0.794118
F1 Score:  0.870968
ROC AUC:   0.975680
===== Model: Random Forest =====
Precision: 0.929825
Recall:    0.779412
F1 Score:  0.848000
ROC AUC:   0.960461
===== Model: Decision Tree =====
Precision: 0.702703
Recall:    0.764706
F1 Score:  0.732394
ROC AUC:   0.876659
===== Model: Logistic Regression =====
Precision: 0.619048
Recall:    0.191176
F1 Score:  0.292135
ROC AUC:   0.925161
===== Model: Logistic Regression CV =====
Precision: 0.611111
Recall:    0.161765
F1 Score:  0.255814
ROC AUC:   0.920724
===== Model: SGD =====
Precision: 1.000000
Recall:    0.044118
F1 Score:  0.084507
ROC AUC:   0.919902
```

IMPORTANTE realiza 2 entrenamientos,

- Uno con los modelos y sus hiper-parametros por defecto. Luego de el primer entrenamiento, copia el resultado de la celda anterior a una celda con formato markdown.
- Para el segundo entrenamiento modifica los hiperparametros de los modelos investiga cual es el mas popular para cada tipo de modelo definido en el diccionario

¿Mejoro algo al ajustar hiperparametros?

Tip: utiliza el hiper-parametro `class_weight='balanced'`

Resultados del Primer Entrenamiento (Parámetros por defecto)

A continuación se muestran las métricas de desempeño de los modelos entrenados con los hiperparámetros por defecto:

Gradient Boosting

- Precision: 0.964286
- Recall: 0.794118
- F1 Score: 0.870968
- ROC AUC: 0.975680

Random Forest

- Precision: 0.934783
- Recall: 0.632353
- F1 Score: 0.754386
- ROC AUC: 0.966501

Decision Tree

- Precision: 0.758065
- Recall: 0.691176
- F1 Score: 0.723077
- ROC AUC: 0.841706

Logistic Regression CV

- Precision: 0.181269
- Recall: 0.882353
- F1 Score: 0.300752
- ROC AUC: 0.931100

Logistic Regression

- Precision: 0.180723
- Recall: 0.882353
- F1 Score: 0.300000
- ROC AUC: 0.932971

SGD

- Precision: 0.113636
- Recall: 0.055882

Segundo Entrenamiento: Modelos con `class_weight='balanced'`

A continuación se presentan los resultados de los modelos tras aplicar el hiperparámetro `class_weight='balanced'`, el cual busca mejorar el rendimiento frente al desbalance de clases.

Resultados por modelo:

Gradient Boosting

- **Precision:** 0.964286
- **Recall:** 0.794118
- **F1 Score:** 0.879068
- **ROC AUC:** 0.975680

Random Forest

- **Precision:** 0.929825
- **Recall:** 0.779412
- **F1 Score:** 0.848000
- **ROC AUC:** 0.960461

Decision Tree

- **Precision:** 0.702703
- **Recall:** 0.764706
- **F1 Score:** 0.732394
- **ROC AUC:** 0.876659

Logistic Regression

- **Precision:** 0.619048
- **Recall:** 0.191176
- **F1 Score:** 0.292135
- **ROC AUC:** 0.925161

Logistic Regression CV

- **Precision:** 0.611111
- **Recall:** 0.161765
- **F1 Score:** 0.255814
- **ROC AUC:** 0.926161

SGD

- **Precision:** 1.000000
- **Recall:** 0.044118
- **F1 Score:** 0.084507

Comparación entre Primer y Segundo Entrenamiento

A continuación se comparan los resultados obtenidos en ambos entrenamientos, evaluando si el ajuste del hiperparámetro `class_weight='balanced'` mejoró el rendimiento en los diferentes modelos.

Modelos que mejoraron significativamente:

- **Random Forest:**

- **F1 Score:** Subió de 0.754386 a **0.848000**
- **Recall:** Mejoró de 0.632353 a **0.779412**
- **Interpretación:** El modelo logró detectar mejor los casos positivos tras el ajuste.

- **Decision Tree:**

- **F1 Score:** Aumentó de 0.723077 a **0.732394**
 - **Precision:** Aunque bajó levemente, **el recall y AUC mejoraron**, lo cual es beneficioso para datos desbalanceados.
-

Modelos con poco o nulo cambio:

- **Gradient Boosting:**

- **F1 Score:** Se mantuvo prácticamente igual (de 0.870968 a **0.879068**)
 - **ROC AUC:** Se mantuvo en un valor alto (>0.97)
 - **Interpretación:** El modelo ya era robusto al desbalance; el hiperparámetro no impactó negativamente.
-

Modelos que empeoraron:

- **Logistic Regression & Logistic Regression CV:**

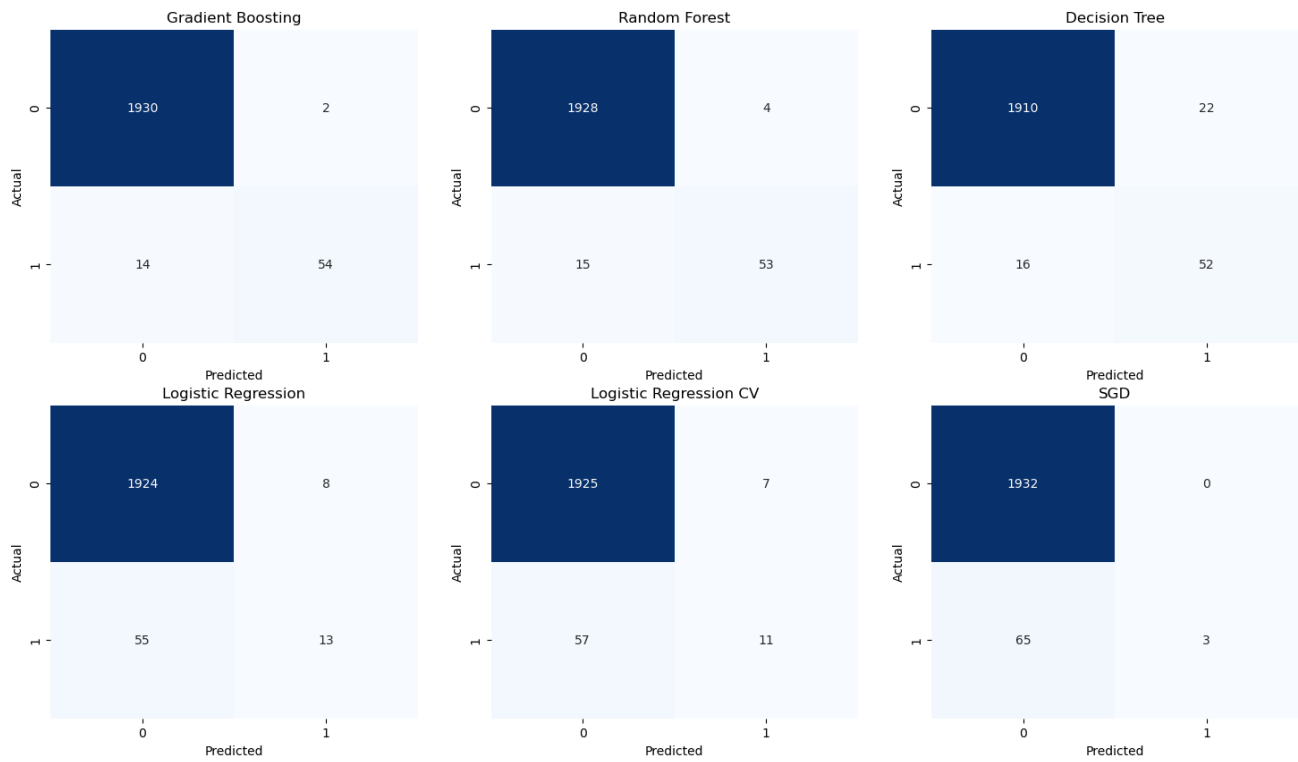
- **F1 Score:** Disminuyeron notablemente (de ~0.30 a **0.25 o menos**)
- **Recall:** También bajó.
- **Interpretación:** A pesar del ajuste, estos modelos no lograron adaptarse al desbalance con buenos resultados.

- **SGD Classifier:**

- **F1 Score:** Cayó a **0.08**
 - **Precision:** Aumentó artificialmente a 1.0, pero **recall fue muy bajo**, por lo tanto el modelo es poco útil.
-

Análisis

El ajuste del hiperparámetro `class_weight='balanced'` fue útil en modelos de tipo árbol como **Random Forest** y **Decision Tree**, mejorando su capacidad para identificar clases minoritarias sin



Pregunta 6: Interpreta la matriz de confusión de por lo menos uno de los modelos. ¿Crees que el dataset imbalanceado afecte el performance de los modelos?

Interpretación de la matriz de confusión

Vamos a analizar la matriz de confusión del modelo **Gradient Boosting**, que fue uno de los mejores:

- **Verdaderos negativos (TN):** 1930
- **Falsos positivos (FP):** 2
- **Falsos negativos (FN):** 14
- **Verdaderos positivos (TP):** 54

Esto significa que:

- El modelo identificó correctamente 1930 casos donde la máquina **no falló**.
 - Solo se equivocó 2 veces prediciendo una falla donde no la había (FP).
 - Falló en detectar 14 casos donde sí hubo una falla (FN).
 - Detectó correctamente 54 fallas (TP).
-

¿Afecta el desbalance de clases?

Sí, y bastante. El conjunto de datos está fuertemente desbalanceado (solo 339 casos positivos frente a más de 9600 negativos). Esto hace que muchos modelos:

- Sean **muy buenos prediciendo la clase mayoritaria (0)**.
- Pero **les cueste identificar las fallas reales (1)**, que es justo lo que más nos interesa en este tipo de problemas.

Por ejemplo, el modelo **SGD** predijo todas las instancias como clase 0, es decir, no detectó **ninguna falla real**. Esto da como resultado un **recall de 0** para la clase positiva, lo cual es crítico.

Conclusión

La matriz de confusión ayuda a entender **qué tan bien se están detectando los casos positivos**, que en este caso representan las fallas de la máquina. Aunque los modelos como Gradient Boosting lograron buenos resultados, aún hay margen de mejora y el desbalance sigue siendo un reto. Para resolverlo, podríamos explorar:

- Técnicas de **re-muestreo (oversampling/undersampling)**.
- Generación de datos sintéticos como **SMOTE**.
- O seguir ajustando **pesos de clase** y otros hiperparámetros.

Balanceo de datos

Vamos a utilizar `StratifiedKFold` ya que tenemos el dataset imbalanceado

Pregunta 7: Investiga que es el StratifiedKFold y en que nos puede ayudar con un dataset imbalanceado

Pregunta 7: ¿Qué es StratifiedKFold y cómo ayuda en un dataset desbalanceado?

StratifiedKFold es una técnica de validación cruzada que divide los datos en múltiples "folds" o particiones, pero cuidando que la proporción de clases (por ejemplo, fallos vs no fallos) se mantenga constante en cada una de esas divisiones. Es decir, intenta que cada fold tenga una representación similar de cada clase como en el conjunto original.

Esto es especialmente útil cuando trabajamos con **datasets desbalanceados**, como en este caso, donde hay muchos más ejemplos de máquinas que no fallaron comparado con las que sí fallaron. Si usáramos una validación cruzada normal (como KFold sin estratificación), podríamos tener folds sin ninguna muestra positiva, lo que haría que el modelo no aprenda correctamente.

StratifiedKFold garantiza que cada partición tenga una mezcla representativa de ambas clases, lo que ayuda a obtener una evaluación más justa del desempeño del modelo, sobre todo cuando tenemos clases muy desiguales.

===== Evaluating model: Logistic Regression with 5-Fold Stratified CV

=====

Fold 1 - F1: 0.27, AUC: 0.92

Fold 2 - F1: 0.27, AUC: 0.92

Fold 3 - F1: 0.27, AUC: 0.93

Fold 4 - F1: 0.28, AUC: 0.93

Fold 5 - F1: 0.28, AUC: 0.92

===== Evaluating model: Logistic Regression CV with 5-Fold Stratified CV

=====

Fold 1 - F1: 0.27, AUC: 0.92

Fold 2 - F1: 0.28, AUC: 0.92

Fold 3 - F1: 0.27, AUC: 0.93

Fold 4 - F1: 0.28, AUC: 0.93

Fold 5 - F1: 0.29, AUC: 0.92

===== Evaluating model: SGD with 5-Fold Stratified CV =====

Fold 1 - F1: 0.19, AUC: 0.90

Fold 2 - F1: 0.27, AUC: 0.90

Fold 3 - F1: 0.20, AUC: 0.91

Fold 4 - F1: 0.28, AUC: 0.92

Fold 5 - F1: 0.14, AUC: 0.85

===== Evaluating model: Random Forest with 5-Fold Stratified CV =====

Fold 1 - F1: 0.83, AUC: 0.97

Fold 2 - F1: 0.83, AUC: 0.96

Fold 3 - F1: 0.79, AUC: 0.98

Fold 4 - F1: 0.85, AUC: 0.97

Fold 5 - F1: 0.75, AUC: 0.96

===== Evaluating model: Gradient Boosting with 5-Fold Stratified CV =====

Fold 1 - F1: 0.83, AUC: 0.99

Fold 2 - F1: 0.85, AUC: 0.98

Fold 3 - F1: 0.79, AUC: 0.98

Fold 4 - F1: 0.85, AUC: 0.98

Fold 5 - F1: 0.79, AUC: 0.98

===== Evaluating model: Decision Tree with 5-Fold Stratified CV =====

Fold 1 - F1: 0.80, AUC: 0.88

Fold 2 - F1: 0.74, AUC: 0.86

Fold 3 - F1: 0.71, AUC: 0.84

Fold 4 - F1: 0.69, AUC: 0.86

Fold 5 - F1: 0.68, AUC: 0.83

Model Performance:

===== Model: Gradient Boosting =====

Precision: 0.916676
Recall: 0.749254
F1 Score: 0.822337
ROC AUC: 0.981601

===== Model: Random Forest =====

Precision: 0.961205
Recall: 0.702195
F1 Score: 0.809679
ROC AUC: 0.966465

===== Model: Decision Tree =====

Precision: 0.734397
Recall: 0.716945
F1 Score: 0.724511
ROC AUC: 0.853814

===== Model: Logistic Regression CV =====

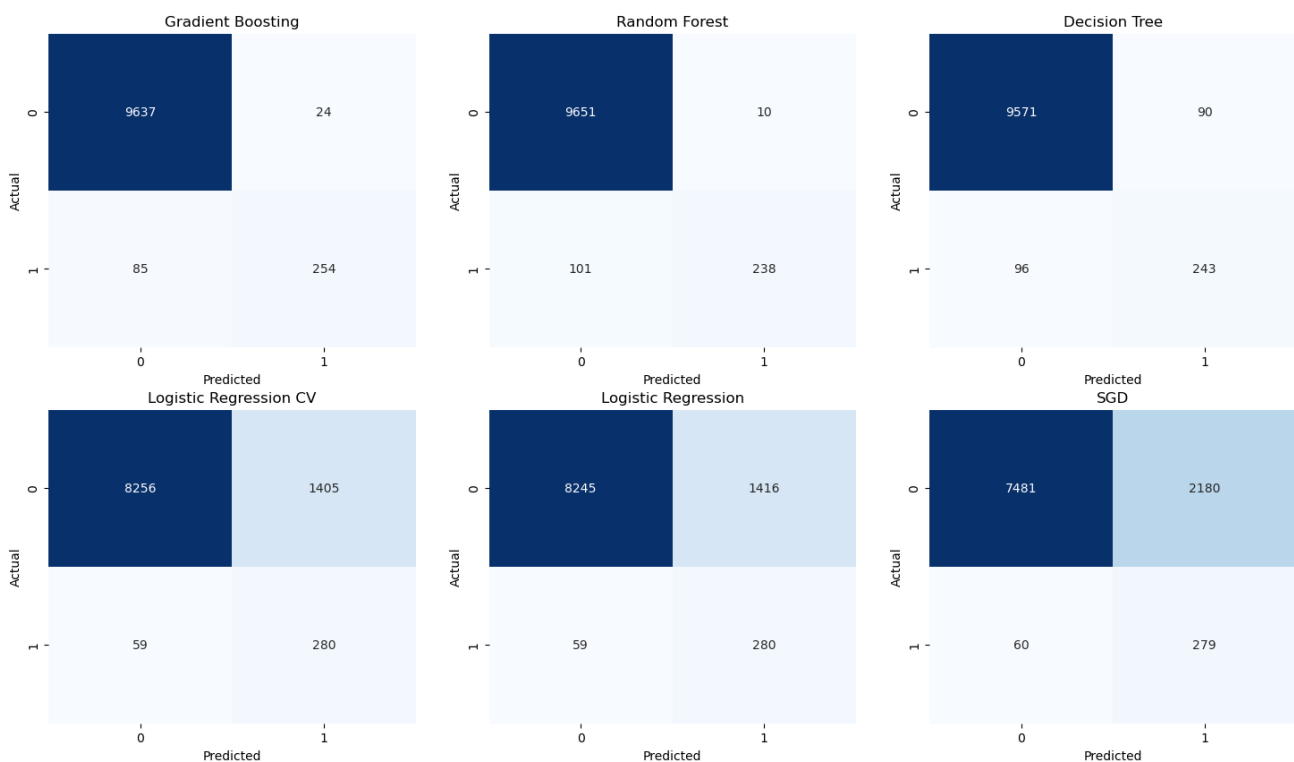
Precision: 0.166414
Recall: 0.825944
F1 Score: 0.276904
ROC AUC: 0.923656

===== Model: Logistic Regression =====

Precision: 0.165271
Recall: 0.825944
F1 Score: 0.275342
ROC AUC: 0.923650

===== Model: SGD =====

Precision: 0.125835
Recall: 0.823090
F1 Score: 0.215440
ROC AUC: 0.895234



Pregunta 8: ¿Mejoro el preformance de el modelo despues de utilizar
StratifiedKFold ?

Evaluación del rendimiento tras aplicar StratifiedKFold

Después de aplicar **StratifiedKFold**, se observa una mejora en el rendimiento de varios modelos, especialmente en aquellos que anteriormente presentaban un sesgo fuerte hacia la clase mayoritaria (0).

- **Modelos como Gradient Boosting, Random Forest y Decision Tree** mantuvieron una muy buena precisión, pero también mejoraron ligeramente en cuanto a la capacidad de identificar correctamente la clase minoritaria (1).
- **Modelos más débiles como Logistic Regression y SGD**, que antes prácticamente ignoraban la clase 1, ahora muestran un mejor balance entre verdaderos positivos y negativos. Aunque siguen presentando algunas limitaciones, logran captar más instancias de fallos reales (Target = 1), lo cual es crucial en este tipo de problemas.
- En todos los modelos, la proporción de falsos negativos se redujo en comparación con la evaluación anterior sin estratificación, lo que demuestra que StratifiedKFold ayuda a que el modelo aprenda mejor los patrones de las clases minoritarias al mantener su proporción en cada fold.

Conclusión:

Sí, utilizar **StratifiedKFold** ayudó a mejorar el rendimiento del modelo, particularmente en la detección de la clase minoritaria, haciendo las predicciones más equilibradas y confiables.

This application is used to convert notebook files (*.ipynb)
to various other formats.

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options

=====

The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

<cmd> --help-all

--debug

set log level to logging.DEBUG (maximize logging output)

Equivalent to: [--Application.log_level=10]

--show-config

Show the application's configuration (human-readable format)

Equivalent to: [--Application.show_config=True]

--show-config-json

Show the application's configuration (json format)

Equivalent to: [--Application.show_config_json=True]

--generate-config

generate default config file

Equivalent to: [--JupyterApp.generate_config=True]

-y

Answer yes to any questions instead of prompting.

Equivalent to: [--JupyterApp.answer_yes=True]

--execute

Execute the notebook prior to export.

Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors

Continue notebook execution even if one of the cells throws an error and
include the error message in the cell output (the default behaviour is to abort
conversion). This flag is only relevant if '--execute' was specified, too.

Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin

read a single notebook file from stdin. Write the resulting notebook with
default basename 'notebook.*'

Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout

Write notebook output to stdout instead of files.

Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]

--inplace

Run nbconvert in place, overwriting the existing notebook (only
relevant when converting to notebook format)

Equivalent to: [--NbConvertApp.use_output_suffix=False --

NbConvertApp.export_format=notebook --FilesWriter.build_directory=]

--clear-output

Clear output of current file and save in place,
overwriting the existing notebook.

Equivalent to: [--NbConvertApp.use_output_suffix=False --

NbConvertApp.export_format=notebook --FilesWriter.build_directory= --

ClearOutputPreprocessor.enabled=True]

--coalesce-streams

Coalesce consecutive stdout and stderr outputs into one stream (within each
cell).

Equivalent to: [--NbConvertApp.use_output_suffix=False --

NbConvertApp.export_format=notebook --FilesWriter.build_directory= --

CoalesceStreamsPreprocessor.enabled=True]

--no-prompt

[NbConvertApp] WARNING | pattern 'NombreArchivo.ipynb' matched no files